

SDLC Agents for Enterprise

AWS Deployment — Scaling Roadmap

Kế hoạch scale-up từ MVP đến production: bắt đầu ~\$388/tháng, tăng dần khi user tăng.
EC2 GPU (Backend + Ollama) + PostgreSQL. SDLC chính chạy Kiro từ client side.

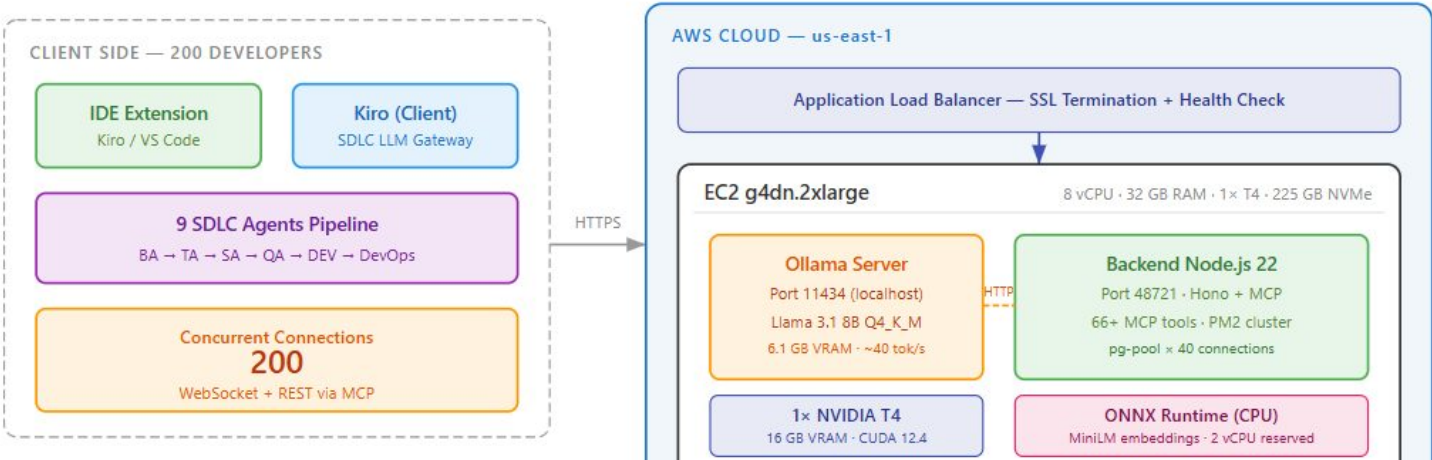
Target: **us-east-1** GPU: **1x NVIDIA T4 (16 GB)** Scale: **MVP → 200 users** Start: **~\$388/mo**

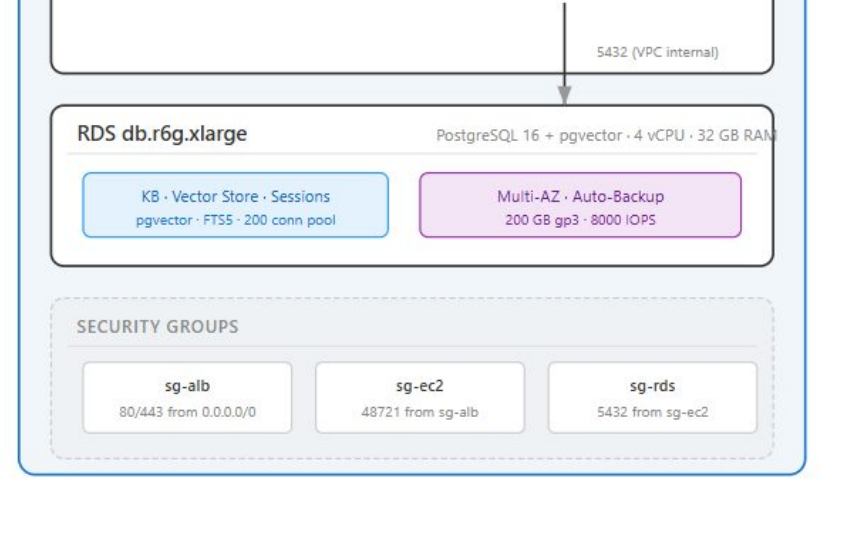
EXECUTIVE SUMMARY

Dự án mới khởi động — không cần chạy full 200 users từ ngày đầu. Kế hoạch 4 phase scale-up dần, mỗi phase tăng resource khi thật sự cần:

- Phase 1 (MVP): ~\$388/tháng** — 1 EC2 g4dn.xlarge all-in-one (Backend + Ollama + PostgreSQL Docker).
- Phase 2 (Early Growth): ~\$571/tháng** — tách PostgreSQL ra RDS.
- Phase 3 (Growth): ~\$1,073/tháng** — upgrade EC2 + thêm ALB.
- Phase 4 (Production): ~\$1,243/tháng** — full architecture 200 users.

Kiến trúc mục tiêu — Phase 4 (Production)





Instance Sizing — Bắt đầu nhỏ, scale dần

CHIẾN LƯỢC: PHASE 1 ALL-IN-ONE → PHASE 2+ TÁCH DẦN

Phase 1 chạy tất cả trên 1 EC2 g4dn.xlarge (Backend + Ollama + PostgreSQL Docker). Chi phí chỉ \$388/mo. Khi user tăng, tách PostgreSQL ra RDS (Phase 2), upgrade EC2 (Phase 3), thêm ALB (Phase 3-4). Mỗi phase chỉ mất 15-30 phút downtime để resize — không cần rebuild.

PHASE	EC2	DB	ALB	USERS	ON-DEMAND
★ 1 — MVP	g4dn.xlarge (4C/16G/T4)	Docker PG (trên EC2)	—	1–20	\$388
2 — Early Growth	g4dn.xlarge (4C/16G/T4)	RDS r6g.large (2C/16G)	—	20–50	\$571
3 — Growth	g4dn.2xlarge (8C/32G/T4)	RDS r6g.large (2C/16G)	ALB	50–100	\$1,073
4 — Production	g4dn.2xlarge (8C/32G/T4)	RDS r6g.xlarge (4C/32G)	ALB	100–200	\$1,243

Phase 1 **\$388**

Phase 2 **\$571**

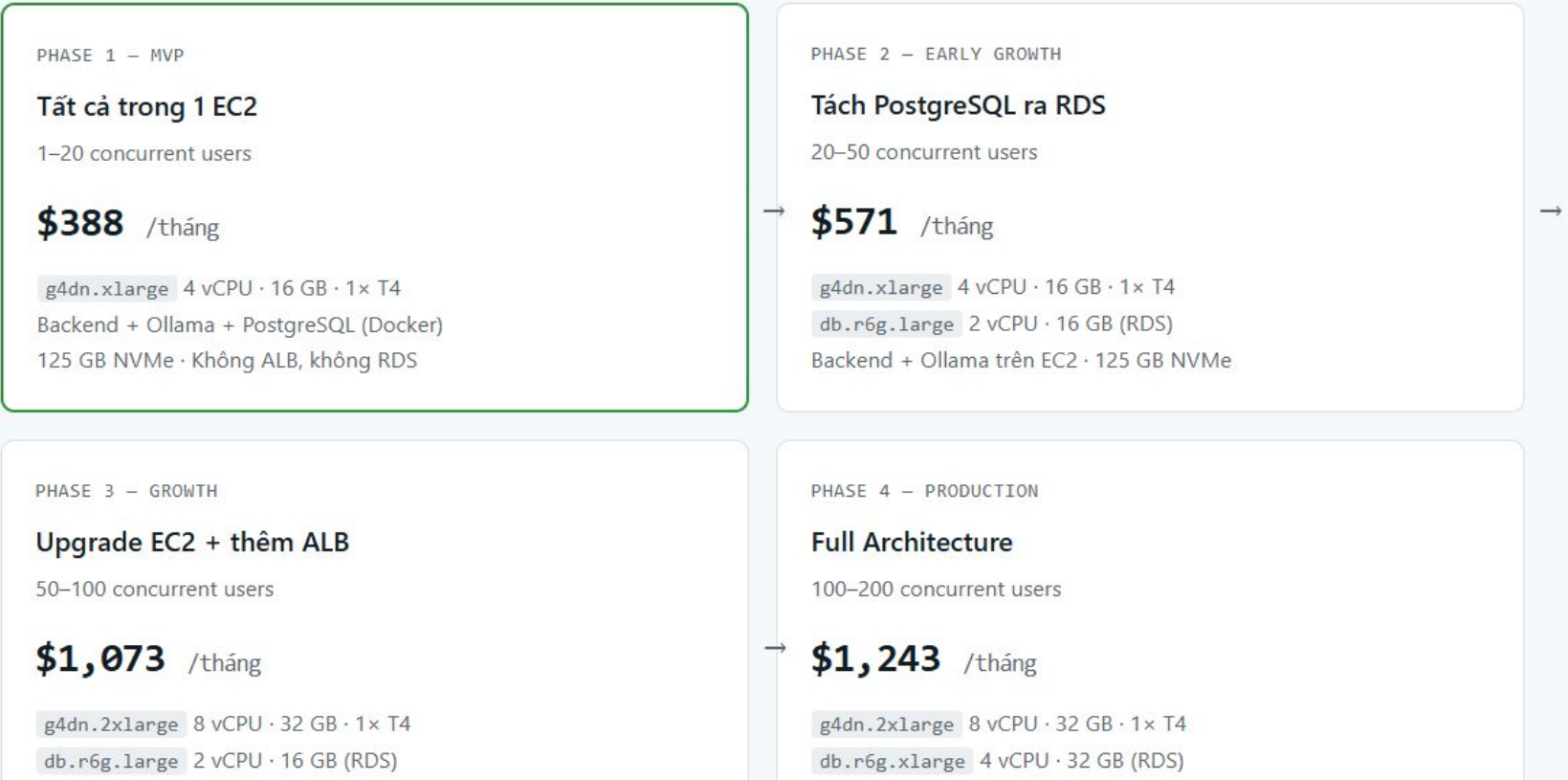
Phase 3 **\$1,073**

Phase 4 **\$1,243**

Capacity per Phase

RESOURCE	PHASE 1	PHASE 2	PHASE 3	PHASE 4
EC2	g4dn.xlarge (4C/16G)	g4dn.xlarge (4C/16G)	g4dn.2xlarge (8C/32G)	g4dn.2xlarge (8C/32G)
DB	Docker PG (shared)	RDS r6g.large (2C/16G)	RDS r6g.large (2C/16G)	RDS r6g.xlarge (4C/32G)
Node.js req/s	~1200/s	~1200/s	~2400/s	~2400/s
DB connections	20 pooled	20 pooled	40 pooled	40 pooled
Ollama concurrency	4-6 requests	4-6 requests	4-6 requests	4-6 requests
GPU VRAM	7.3/16 GB (54%)	7.3/16 GB (54%)	7.3/16 GB (54%)	7.3/16 GB (54%)
Max users	20	50	100	200

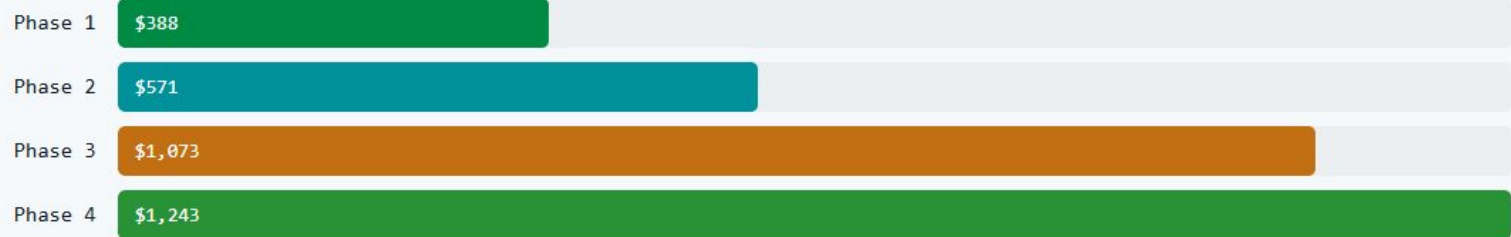
Kế hoạch Scale-Up — 4 Phase



+ ALB (SSL + health check) · 225 GB NVMe

ALB · 200 GB RDS gp3 · 225 GB NVMe

SO SÁNH CHI PHÍ ON-DEMAND



PHASE	EC2	RDS	ALB	STORAGE	TOTAL/MO	USERS
★ 1 — MVP	g4dn.xlarge \$384	Docker (trên EC2) \$0	—	EBS 125 GB \$10	\$388	1–20
2 — Early Growth	g4dn.xlarge \$384	db.r6g.large \$164	—	EBS \$10 + RDS \$16	\$571	20–50
3 — Growth	g4dn.2xlarge \$838	db.r6g.large \$164	ALB \$32	EBS \$18 + RDS \$16	\$1,073	50–100
4 — Production	g4dn.2xlarge \$838	db.r6g.xlarge \$329	ALB \$32	EBS \$18 + RDS \$16	\$1,243	100–200

RESERVED INSTANCE — TIẾT KIỆM ~37-40% KHI ỔN ĐỊNH

PHASE	ON-DEMAND	RESERVED 1Y	SPOT + RI	TIẾT KIỆM
1 — MVP	\$388	\$258	—	-\$130/mo
2 — Early Growth	\$571	\$386	—	-\$185/mo
3 — Growth	\$1,073	\$700	\$517	-\$373/mo
4 — Production	\$1,243	\$752	\$546	-\$697/mo

GPU ON-DEMAND — STRATEGIES · GIỜ CHẠY = GIỜ LÀM VIỆC



Scheduled Scaling

Chỉ bật GPU trong giờ làm việc (8h/ngày, 22 ngày/tháng). Tự động start/stop bằng EventBridge + Lambda. Phù hợp Phase 1-2.

Tiết kiệm 66-72%



Auto-Stop on Idle

GPU tự stop khi CPU/GPU idle > 15 phút. CloudWatch alarm kích hoạt Lambda. Phù hợp dev/test environments.

Tiết kiệm 50-60%



Spot Instances

Sử dụng EC2 Spot (60-70% off On-Demand). Tự động restart khi bị reclaim. Kết hợp với On-Demand base capacity.

Tiết kiệm 60-70%

PHASE	ON-DEMAND (24/7)	SCHEDULED (8H/NGÀY)	AUTO-STOP IDLE	SPOT + SCHEDULED	TIẾT KIỆM
1 — MVP	\$388	\$129	\$194	\$39	-\$349/mo
2 — Early Growth	\$571	\$313	\$285	\$94	-\$477/mo
3 — Growth	\$1,073	\$357	\$537	\$107	-\$966/mo
4 — Production	\$1,243	\$414	\$622	\$124	-\$1,119/mo

Khuyến nghị: Bắt đầu với **Scheduled Scaling** (Phase 1-2). Khi có usage data thật, chuyển sang **Spot + Scheduled** (Phase 3-4) để tiết kiệm tối đa. Kết hợp Auto-Stop on Idle như fallback.

CHIẾN LƯỢC	LỊCH TRÌNH	GIỜ GPU/THÁNG	CHI PHÍ GPU	PHÙ HỢP
On-Demand 24/7	Luôn chạy	730h	\$384 (g4dn.xlarge)	Production 24/7
Scheduled (Business)	8h/ngày × 22 ngày	176h	\$93	Dev team giờ hành chính
Scheduled (Extended)	12h/ngày × 22 ngày	264h	\$139	Team多个 timezone
Auto-Stop Idle	Khi idle > 15 phút	~365h (50% uptime)	\$192	Sporadic usage
Spot (Preemptible)	Luôn chạy (tự reclaim)	~650h (90% success)	\$115	Non-critical workloads
Spot + Scheduled	8h/ngày × Spot	~158h	\$28	Best cost optimization

IMPLEMENTATION — AUTO-START/STOP SCRIPTS

```
#!/bin/bash
# EventBridge Rule – Start GPU khi có request
# Cron: Monday-Friday 8:00 AM UTC+7 → 1:00 AM UTC
aws events put-rule \
  --name "start-gpu-business-hours" \
  --schedule-expression "cron(0 1 ? * MON-FRI *)"
```

```
# Lambda: Start EC2 + Wait for ready
def lambda_handler(event, context):
    ec2 = boto3.client('ec2')
    ec2.start_instances(InstanceIds=['i-0abc123...'])
    waiter = ec2.get_waiter('instance_running')
    waiter.wait(InstanceIds=['i-0abc123...'])
    # Trigger ECS service update to attach GPU
    return {"status": "started"}
```

AUTO-STOP ON IDLE — CLOUDWATCH ALARM

```
#!/bin/bash
# Tạo CloudWatch Alarm cho GPU idle
aws cloudwatch put-metric-alarm \
  --alarm-name "gpu-idle-stop" \
  --metric-name "GPUUtilization" \
  --namespace "AWS/EC2" \
  --statistic "Average" \
  --period 900 \
  --threshold 5 \
  --comparison-operator "LessThanThreshold" \
  --dimensions "Name=InstanceId,Value=i-0abc123..." \
  --alarm-actions "arn:aws:lambda:us-east-1:123456789:function:stop-gpu-instance"

# Lambda: Stop EC2 khi idle
def lambda_handler(event, context):
    ec2 = boto3.client('ec2')
    ec2.stop_instances(InstanceIds=['i-0abc123...'])
    return {"status": "stopped"}
```

Monitoring: Theo dõi GPU utilization qua CloudWatch. Nếu > 80% thường xuyên → cần upgrade instance. Nếu < 30% → cân nhắc giảm GPU hoặc tăng scheduled hours.

KHI NÀO NÊN UPGRADE LÊN PHASE TIẾP THEO?

Phase 1 → 2

CPU EC2 > 70% thường xuyên · DB cần được backup riêng · Cần point-in-time recovery

Phase 2 → 3

Concurrent users > 50 · Response time > 500ms · Cần SSL certificate managed · Cần health check tự động

Phase 3 → 4

Cần chuyển sang 100% DB managed · Cần chuyển sang RDS

Khi nào mua Reserved?

Cần dự đoán traffic ổn định, dự đoán được thời gian hoạt động

Concurrent users > 100 · DB queries slow · Cần more RAM cho pgvector · DB connection pool near limit

Sau 3-6 tháng chạy ổn định, xác nhận instance type phù hợp → mua 1Y RI để tiết kiệm 37-40%

Chiến lược tối ưu: Bắt đầu Phase 1 (~\$388/mo). Sau 1-2 tháng, khi có data thật về usage patterns, quyết định upgrade. Mỗi phase chỉ cần resize instance (15 phút downtime) hoặc thêm service mới — không cần rebuild từ đầu.

Quy trình triển khai — Phase 1 (MVP)

- 1

Tạo EC2 Instance — All-in-One
Launch g4dn.xlarge AMI Ubuntu 22.04, public subnet. Mở port 22 (SSH), 48721 (backend). Attach IAM role (SSM + CloudWatch). Chi phí: \$384/mo.
- 2

Cài đặt NVIDIA Driver + Docker
`sudo apt update && sudo apt install -y nvidia-driver-550 docker.io docker-compose-plugin` · Reboot · Verify `nvidia-smi`.
- 3

Deploy bằng Docker Compose
Clone repo → `docker compose -f docker-compose.yml up -d` lên cả 3 service: PostgreSQL 16 + pgvector, Backend Node.js, ONNX Runtime. Ollama cài riêng bare-metal.
- 4

Cài Ollama + Pull Model
`curl -fsSL https://ollama.com/install.sh | sh` · `ollama pull llama3.1:8b` (~4.9 GB). Ollama bind `127.0.0.1:11434`.
- 5

SSL + Security
Dùng Let's Encrypt hoặc ACM (nếu dùng CloudFront). UFW firewall: chỉ mở 22 + 48721. Enable EBS encryption.
- 6

Verify + Monitoring
Test health check `/health`. CloudWatch agent: CPU, RAM, GPU metrics. Billing alarm \$500.

UPGRADE PATH — PHASE 1 → 2 → 3 → 4

Mỗi phase upgrade chỉ cần thao tác trên AWS Console hoặc Terraform:

- Phase 1 → 2

Tạo RDS db.r6g.large → migrate DB → cập nhật `.env` DB_HOST → 停 container PostgreSQL trên EC2
- Phase 2 → 3

Stop EC2 → change instance type g4dn.xlarge → g4dn.2xlarge → Start. Tạo ALB + target group.

Phase 3 → 4

RDS: Modify db.r6g.large → db.r6g.xlarge (downtime 5 phút). Tăng RDS storage nếu cần.

Backup Strategy

Phase 1: EBS snapshots hàng ngày. Phase 2+: RDS automated backup 7 ngày + cross-region snapshot.

Terraform IaC — Phase 1 (MVP)

```
PHASE1.TF - ALL-IN-ONE
provider "aws" { region = "us-east-1" } # — Security Group — resource "aws_security_group" "sdlc_mvp" {
name_prefix = "sdlc-mvp-" ingress { from_port=48721 to_port=48721 protocol="tcp" cidr_blocks=["0.0.0.0/0"] description
= "Backend API" } ingress { from_port=22 to_port=22 protocol="tcp" cidr_blocks=["YOUR_IP/32"] description = "SSH" }
egress { from_port=0 to_port=0 protocol="-1" cidr_blocks=["0.0.0.0/0"] } } # — EC2 All-in-One (Backend + Ollama +
PostgreSQL Docker) — resource "aws_instance" "sdlc_mvp" { ami = "ami-0c7217cdde317cfec" # Ubuntu 22.04 instance_type
= "g4dn.xlarge" # 4 vCPU, 16 GB, 1x T4 key_name = var.key_name vpc_security_group_ids =
[aws_security_group.sdlc_mvp.id] root_block_device { volume_size = 125 volume_type = "gp3" encrypted = true }
user_data = <<-EOF #!/bin/bash apt update && apt upgrade -y apt install -y nvidia-driver-550 docker.io docker-compose-
plugin git curl -fsSL https://ollama.com/install.sh | sh ollama pull llama3.1:8b systemctl enable ollama docker #
Clone + start services git clone https://github.com/your-org/sdlc-agents.git /opt/sdlc cd /opt/sdlc && docker compose
-f docker-compose.yml up -d EOF tags = { Name = "sdlc-mvp-all-in-one" } } # — Outputs — output "public_ip" { value
= aws_instance.sdlc_mvp.public_ip } output "backend_url" { value = "http://${aws_instance.sdlc_mvp.public_ip}:48721" }
```

Phase 2+ cần thêm: RDS PostgreSQL, ALB, VPC private subnets. Xem section Kiến trúc 2-Tier ở trên khi upgrade. Terraform code cho Phase 4 đã có ở version trước của proposal.

Environment Configuration — Phase 1

```
.ENV.PROD
# — Server — PORT=48721 HOST=0.0.0.0 NODE_ENV=production # — Database (Phase 1: Docker PostgreSQL trên cùng EC2)
— # Phase 2+: đổi DB_HOST thành RDS endpoint DB_DRIVER=postgres DB_HOST=postgres # Docker service name (Phase 1) #
DB_HOST=sdlc-agents-db.xxxx.us-east-1.rds.amazonaws.com # Phase 2+ DB_PORT=5432 DB_NAME=sdlc_agents DB_USER=sdlc_user
DB_PASSWORD=${DB_PASSWORD} DB_SSL=false # Phase 1: Docker internal, no SSL. Phase 2+: true DB_POOL_MAX=20 # Phase 1:
20就够. Phase 3+: increase to 40 # — LLM (Local Ollama - code analysis only) —
OLLAMA_BASE_URL=http://127.0.0.1:11434 OLLAMA_MODEL=llama3.1:8b # — Client-side LLMs (SDLC via Kiro from client) —
# Anthropic/OpenAI keys used by Kiro gateway on client, not server-side ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
OPENAI_API_KEY=${OPENAI_API_KEY} # — ONNX (CPU) — ONNX_EMBEDDING_MODEL=Xenova/paraphrase-multilingual-MiniLM-L12-
v2 # — Security — JWT_SECRET=${JWT_SECRET} ENCRYPTION_KEY=${ENCRYPTION_KEY}
```

Rủi ro & Mitigation

RỦI RO	MỨC ĐỘ	MITIGATION
Phase 1: PostgreSQL trên Docker — data loss	Cao	EBS snapshot hàng ngày + Docker volume backup. Upgrade Phase 2 sớm khi có data quan trọng.
Phase 1: Single EC2 — SPOF	Trung bình	EBS-backed (data survives stop/start). ASG reconnect nếu instance fail. Phase 3+ tách riêng.
Ollama crash khi load tăng	Thấp	systemd auto-restart; backend request queuing với timeout 30s; LLM chỉ cho code analysis, không critical path.
GPU instance capacity (Spot)	Trung bình	Phase 1-2: On-demand (\$384-571/mo, affordable). Spot chỉ dùng Phase 3+ khi cần tiết kiệm.
Client-side LLM API downtime (Kiro)	Cao	Kiro gateway có built-in failover; fallback Anthropic/OpenAI trực tiếp; local Ollama làm emergency backup.

Next Steps

- 1

Duyệt Phase 1 — chi phí \$388/mo

Xác nhận g4dn.xlarge, region us-east-1. Triển khai ngay, không cần commit lớn.
- 2

Tạo AWS Account + EC2

Đăng ký AWS, tạo IAM user, setup MFA + billing alarm \$500. Launch EC2 g4dn.xlarge.
- 3

Triển khai Terraform Phase 1

`terraform init && terraform apply`. Cài NVIDIA driver + Docker + Ollama.
- 4

Deploy + Test

Docker Compose lên PostgreSQL + Backend. Verify health check. Kết nối Kiro client từ xa.
- 5

Monitor 1-2 tháng → Quyết định upgrade

Theo dõi CPU, RAM, DB size, concurrent users. Khi đạt trigger → upgrade lên Phase 2.

SDLC Agents for Enterprise – AWS Deployment Proposal (200 Users) · Generated Sep 2026